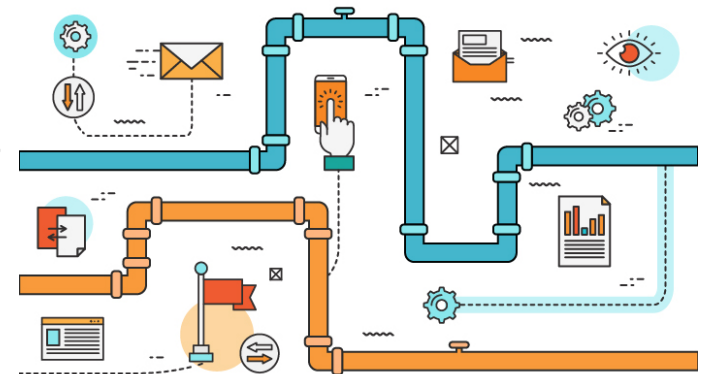


Infrastructure as Code

Change Management Pipelines for Infrastructure

Ron C. Chiang

University of St. Thomas



Agenda

- Change Management Pipelines
- AWS services
 - CodeBuild
 - CodeDeploy
 - CodePipeline

Deployment Pipeline

- A deployment pipeline is used to **manage** the deployment and **testing** of software using a set of **validation stages** whenever there is a **code change**.



Pipeline Benefits

- **Software is automatically built and tested.**
 - Ready to go to production at any time.
- **Making changes to software is much easier.**
 - Every change is tested the same way
 - Easier to automatically deploy changes versus using manual methods
 - Reinforces practice of using small batch sizes
- **Compliance and governance tasks are easier.**
 - Traceability improved and tests are enforced.
 - Reduces need for heavy change control tasks.

Real World Pipeline Development

- Some companies hire people to focus exclusively on pipeline development work.
 - CI/CD engineers
 - Software Delivery engineers
 - DevOps Engineer
 - Release Engineer
 - Build Engineer
- Software engineers today have to know how to build pipelines.

Basic Pipeline Design

- **Local development stage**
 - Developers run and test software on their local workstation
 - Natively, virtual machine (vagrant), container
 - **Linting**: a static code analysis process which identifies common syntax and style defects using a linter application.
 - **Unit testing**: functional tests performed at a small, modular level.

Basic Pipeline Design

- **Build stage**

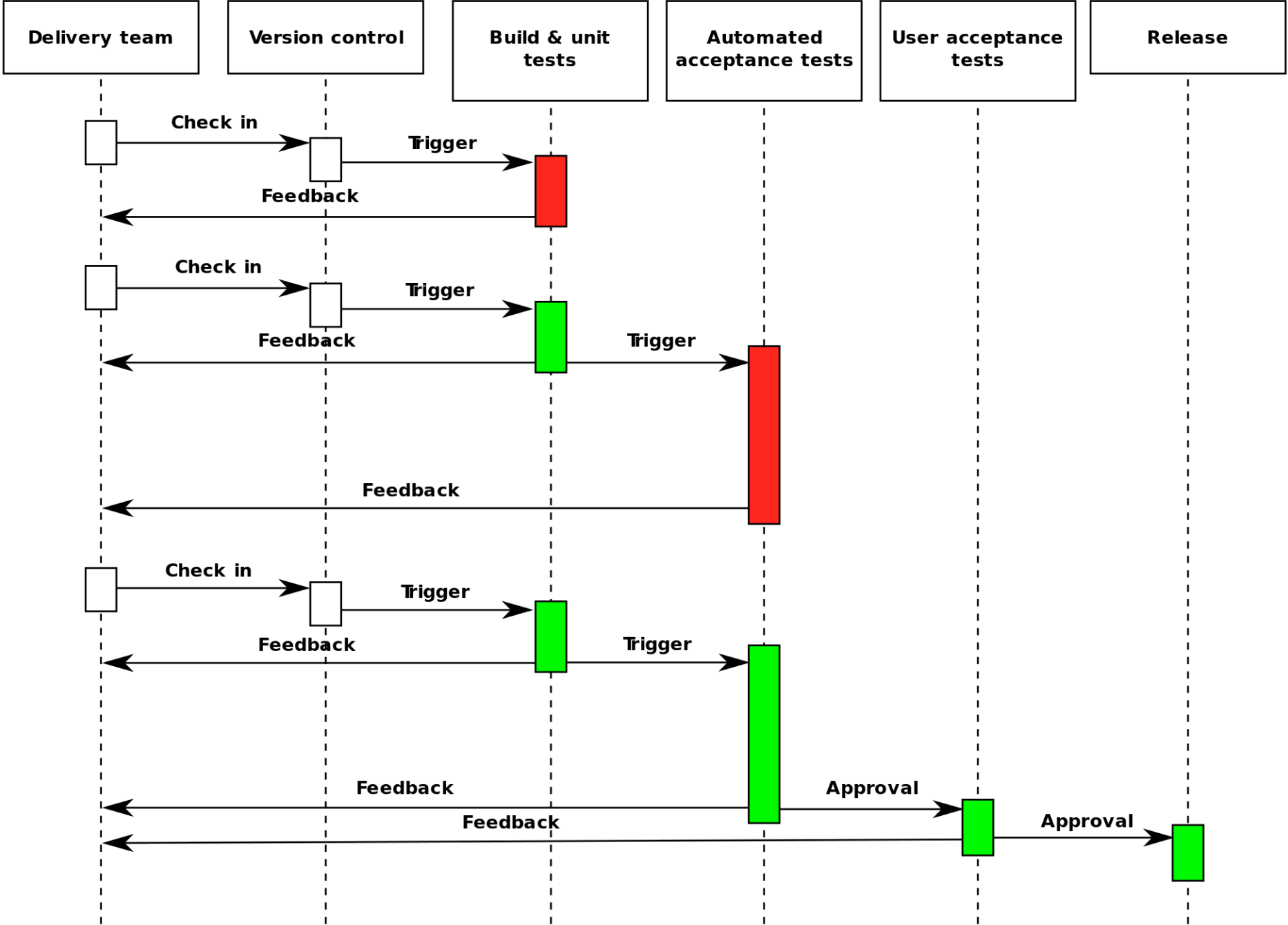
- Usually triggered when a developer checks new code into a VCS
- Linting process
- Code compilation (when necessary)
- Unit tests
- Possible local execution tests
- Package code and store in an **artifact repository**

Basic Pipeline Design

- **Testing stage**
 - Code artifact is deployed to a testing environment
 - Automated API and/or GUI tests are run against environment
 - Both integration and end-to-end testing
 - May leverage a grid-based testing solution
 - Non-functional testing (service resiliency)
 - Performance testing
 - Manual testing (pipeline is stopped and manually retriggered)

Basic Pipeline Design

- **Production deployment stage/ release**
 - Code artifact is deployed to production environment
 - May be first applied to a staging environment for further testing before production
 - Automated smoke tests are run against deployment to validate application working properly.

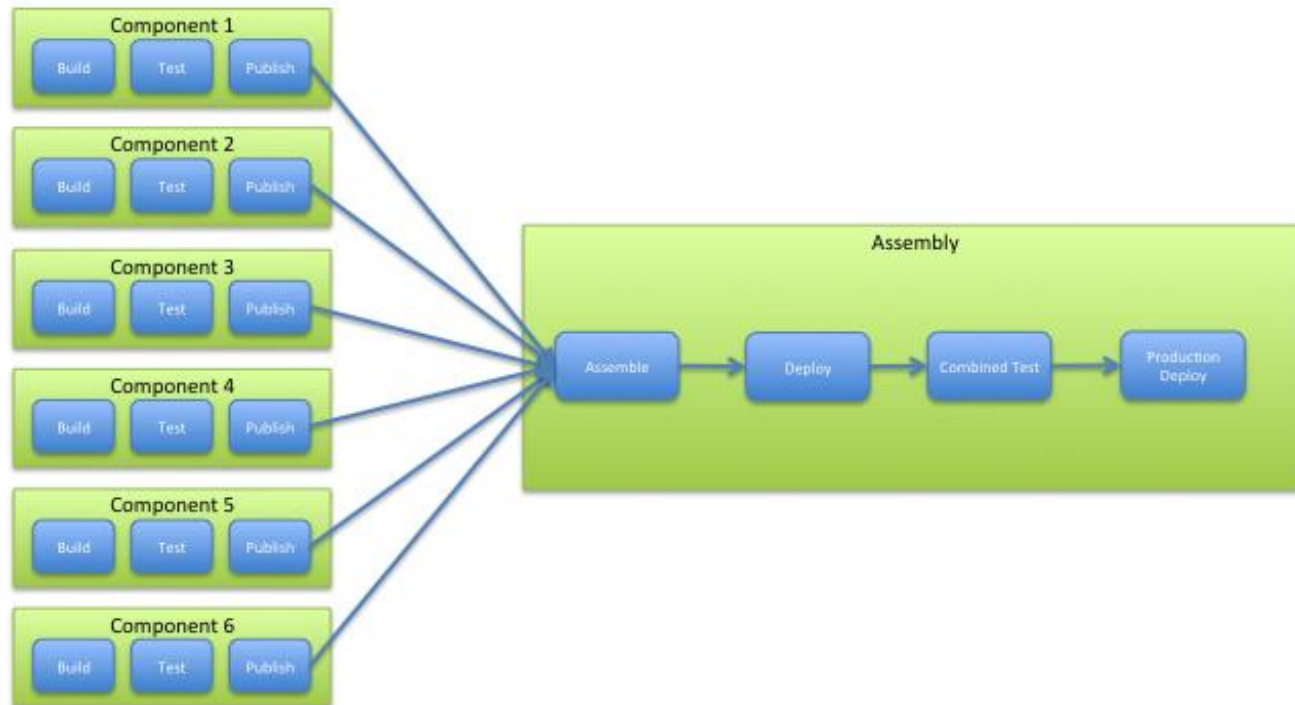


Deployment Pipeline Practices

- **Prove production-readiness for every change**
 - All changes are potential releasable.
- **Only build your binaries once.**
 - Don't keep re-compiling or packaging app at each stage to minimize introduction of configuration inconsistencies.
 - Store application artifacts in a repository (example: Artifactory)
- **Deploy the same way to every environment.**
 - Use the same script to deploy to a dev laptop, a test virtual machine, and a public cloud instance.
 - Store environment-specific configurations in separate files.
- **If any part of the pipeline fails, stop the line.**
 - If a deployment to an environment fails, the whole team owns that failure and every effort should be taken to fix it.

Fan-in Pipeline Pattern

- Multiple pipelines may feed a single assembly or deployment pipeline.



AWS CodeStar



- A cloud development environment which allows developers to write, build, and deploy applications.
 - Cloud9 (not accepting new customers)
 - CodeCommit (not accepting new customers)
 - CodeBuild
 - CodeDeploy
 - CodePipeline

CI/CD pipeline infrastructure

- Building and maintaining CI/CD infrastructure is challenging:
 - Install and update CI server (Jenkins, Bamboo, etc)
 - Manage CI server settings
 - Install and update plugins
 - Create, update, and scale build node servers
 - May need multiple types of build nodes for various jobs
- Don't forget about backups and security!
- Companies oftentimes have dedicated engineers and teams to focus on supporting this infrastructure.
- AWS can handle many of these challenges for us.

CodeBuild



- Managed continuous integration service which allows you to compile code, run unit tests, and create deployment artifacts.
 - No need to provision or manage build servers.
 - Automatically scales computing resources to meet demand.
 - Use pre-packaged environments or bring your own.
 - Java, Python, Node.js, Ruby, Go, Android, .Net Core, Docker
- Billed by the minute for time to complete your build and compute resources used.
 - Free tier: 100 build minutes of build.general1.small instance/month.

CodeBuild

- CodeBuild uses a **build project** to define how to run a build.
 - Defines where the source code is located.
 - What *build environment* to use
 - Set of build commands to run
 - Where to store the build output artifacts
- A **build environment** specifies:
 - Operating system
 - Programming language runtime
 - Tools
- During a build, a build environment is created and source code is downloaded into the environment.

CodeBuild

- A build specification (build spec) defines the commands which CodeBuild uses to run the build.
- We use a YAML file called `buildspec.yml` located within the root directory of the source code repository to store the build spec.

CodeBuild buildspec.yml

version: 0.2

env:

variables:

key: "value"

phases:

install:

runtime-versions:

java: openjdk8

pre_build:

commands:

- echo Nothing to do in the pre_build phase...

build:

commands:

- echo Build started on `date`

- mvn install

finally:

- echo Run this even if the build commands fail.

artifacts:

files:

- target/myApp.jar

name: myApp.jar-\$(date +%Y-%m-%d)

CodeBuild buildspec.yml

- buildspec.yml file always starts with a version

version: 0.2

- You can set environment variables during the build:

env:

variables:

key: "value"

CodeBuild buildspec.yml

- The build process is partitioned into one or more phases:
 - **install**: used for installing run-time packages into the build environment
 - **pre_build**: run command before the build such as installing node dependencies
 - **build**: commands for the actual build process
 - **post-build**: commands run after the build such as packaging output artifacts into a WAR or zip file
- Commands are shell script statements, and the build process will end if any of the commands fail.
- Install runtime versions to support application build requirements: Java, Python, Ruby, etc.
- The finally block of commands will always run even if any of the build commands fail.

phases:

install:

runtime-versions:

java: openjdk8

build:

commands:

- echo Build started on `date`
- mvn install

finally:

- echo Run this even if the build commands fail.

CodedBuild buildspec.yml

Runtime name	Version/versions	How to specify in the buildspec file
android	28	<code>android: 28</code>
docker	18	<code>docker: 18</code>
dotnet	2.2	<code>dotnet: 2.2</code>
golang	1.12	<code>golang: 1.12</code>
nodejs	8, 10	<code>nodejs: 8, nodejs: 10</code>
java	openjdk8, openjdk11	<code>java: openjdk8, java: openjdk11</code>
php	7.3	<code>php: 7.3</code>
python	3.7	<code>python: 3.7</code>
ruby	2.6	<code>ruby: 2.6</code>

CodeBuild buildspec.yml

- The artifacts section describes where the output artifacts are located and optionally what to name these artifacts after copying.
- Artifacts are stored in an S3 bucket.

artifacts:

files:

- target/myApp.jar

name: myApp.jar-\$(date +%Y-%m-%d)

CodeBuild Hands-on



- Goal: build a simple Java application using CodeBuild
- Setup a GitHub repo that CodeBuild can work with:
 - Click the following link and fork a copy of a Java GitHub project into your personal account:

<https://github.com/rc1c/java-project>

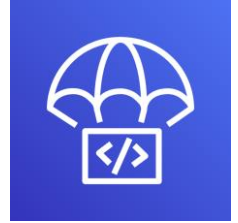
- Create an output artifact S3 bucket.

CodeBuild Hands-on



- Create a build project.
- Start a build and run Apache Ant (a Java build manager).
- Review buildspec.yml file.
 - ant: Apache Ant is a Java build manager
 - Run unit tests, compile Java code, copy artifact to S3
- Update the source code repository and trigger an automated build.
- We can automate this entire configuration using CloudFormation 😊
 - <https://seis665-public.s3.amazonaws.com/app-codebuild-template.json>

CodeDeploy



- A service which automates application code deployments to:
 - EC2 instances
 - Lambda functions
 - Containers (ECS)
 - On-premise servers
- Benefits:
 - Rapidly release new features.
 - Avoid downtime during application deployment.
 - Automate deployments and eliminate manual steps.
 - Scales deployments from a single instance to thousands.

CodeDeploy

- Overall process:
 - Tell CodeDeploy about your application, how it should be deployed, and where it should be deployed to.
 - Add a special configuration file (appspec.yml) to your application bundle which describes how to install the software.
 - Create a deployment which initiates the deployment of the application bundle to a set of compute resources.

CodeDeploy

- Key components:
 - **Application:** code components you want to deploy.
 - **Compute platform:** where you want to deploy application code to (EC2, Lambda, ECS...)
 - **Deployment group:** a set of individual compute instances (EC2)
 - **Deployment type:** the method used to make the latest application version available.

CodeDeploy

- Two deployment types:
 - In-place deployment
 - OneAtATime (Rolling release)
 - HalfAtATime (Rolling release/Min in service)
 - AllAtOnce (Big Bang release)
 - Blue/green

CodeDeploy

- AppSpec file:
 - YAML file which describes how to install the application on the compute platform.
 - Maps application files to destinations on the instance.
 - Specifies custom permissions for deployed files.
 - Specifies scripts to run during stages of the deployment process.

CodeDeploy appspec.yml

version: 0.0

os: linux

files:

- source: myApp.jar
destination: /var/app/myApp.jar

permissions:

- object: /var/app
group: wheel
mode: 440
type:
 - directory

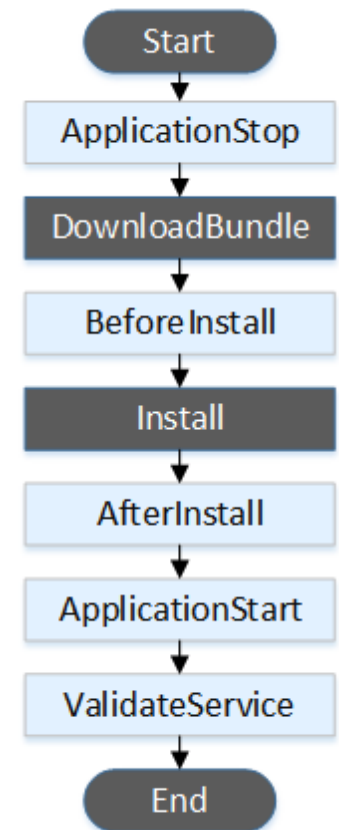
hooks:

AfterInstall:

- location: scripts/run_tests.sh
timeout: 90

CodeDeploy appspec.yml

- Hooks describe lifecycle events which occur during the deployment process.
 - Events differ based on compute platform and deployment type.
 - We can specify scripts to run at each event.
 - Install application dependencies
 - Configure application settings
 - Run post-installation smoke tests



CodeDeploy Hands-on



- Goal: Deploy and application update to a simulated production environment.
- Deploy production environment stack:
 - <https://seis665-public.s3.amazonaws.com/app-prod-env-template.json>
- Upload website.zip file to S3 artifact bucket in stack using encryption.
 - <https://seis665-public.s3.amazonaws.com/website.zip>
- Test access to production ALB
- Create CodeDeploy resources:
 - Application, Deployment Group, Deployment
- Watch the deployment process and lifecycle events.
- Of course we can always build this configuration using code!
 - <https://seis665-public.s3.amazonaws.com/app-codedeploy-template.json>

CodePipeline



- A fully managed continuous delivery service.
 - Check in code to source control.
 - Build software artifacts.
 - Automatically deploy artifacts to compute environment.
- Basically automates the integration of CodeCommit/GitHub, CodeBuild, and CodeDeploy.
- Pricing: \$1 month/pipeline (first month free)

CodePipeline

- Pipeline architecture:
 - A pipeline consists of a set of stages executed sequentially.
 - Each stage consists of a set of actions.
- Example stages:
 - **Source**: Retrieve source code from repository (CodeCommit, GitHub, S3).
 - **Test**: Run unit test suite.
 - **Build**: Compile and/or package source code into output artifact.
 - **Deploy**: Install artifact in compute environment.

CodePipeline

- Stage action types:
 - Source, Build, Test, Deploy, Approval, Invoke
- Each action type supports a specific set of providers:
 - Source: S3, CodeCommit, GitHub
 - Build: CodeBuild, CloudBees, Jenkins, TeamCity
 - Test: CodeBuild, Device Farm, Jenkins, Runscope...
 - Deploy: S3, CloudFormation, CodeDeploy...

CodePipeline

- Technical constraints:
 - Pipeline must contain at least two stages.
 - First stage must contain source actions (i.e., must retrieve source code).
 - At least one of the stages must not contain a source action.
 - Stage names must be unique.

Infrastructure Pipelines

- CI/CD pipelines aren't just used to build software, we also use them to build infrastructure.
- Two general use cases:
 - Automate testing of our infrastructure.
 - A change to a configuration management or infrastructure definition script triggers a pipeline.
 - Automate deployment of infrastructure and our software.

Infrastructure Pipeline

- CloudFormation template pipeline example:
 - Lint CloudFormation template to validate syntax
 - Deploy CloudFormation template
 - Wait for AWS resources to be built
 - Test AWS resources to verify that required resources are installed and working properly
 - Terminate AWS resources
 - Publish new template

CodePipeline Hands-on



- Create GitHub personal access token
- Fork python-project repository:
 - <https://github.com/rclc/python-project.git>
- Review changes to CodeBuild template code
 - Source and Artifacts are managed by CodePipeline
- Deploy CodePipeline template
 - <https://seis665-public.s3.amazonaws.com/app-codepipeline-template.json>
- Watch pipeline execute stages and test application deployment.